

(/)

Spring @Component Annotation

Last updated: May 18, 2023

Written by: Jordan Simpson (<https://www.baeldung.com/author/jordansimpson>)

Spring (<https://www.baeldung.com/category/spring>) +

Spring Annotations (<https://www.baeldung.com/tag/spring-annotations>) **Spring DI** (<https://www.baeldung.com/tag/spring-di>)

Get started with Spring 5 and Spring Boot 2, through the reference *Learn Spring* course:

>> LEARN SPRING (</ls-course-start>)

1. Overview

In this tutorial, we'll take a comprehensive look at the Spring *@Component* (</spring-component-repository-service>) annotation and related areas. We'll see the different ways we can integrate with some core Spring functionality and how to take advantage of its many benefits.

Further reading:

Spring Core Annotations (</spring-core-annotations>)

Explore the most commonly used Spring core annotations.

Read more (</spring-core-annotations>) →

Spring Bean Annotations (</spring-bean-annotations>)

Learn how and when to use the standard Spring bean annotations - `@Component`, `@Repository`, `@Service` and `@Controller`.

Read more (</spring-bean-annotations>) →

Spring Web Annotations (</spring-mvc-annotations>)

Explore the main annotations you can use to handle HTTP requests and responses in Spring MVC.

Read more (</spring-mvc-annotations>) →

2. Spring *ApplicationContext*

Before we can understand the value of `@Component`, we first need to understand a little bit about the Spring `ApplicationContext` (/spring-application-context).

Spring `ApplicationContext` is where Spring holds instances of objects that it has identified to be managed and distributed automatically. These are called beans.

Some of Spring's main features are bean management and the opportunity for dependency injection.

Using the Inversion of Control principle (/inversion-control-and-dependency-injection-in-spring), **Spring collects bean instances from our application and uses them at the appropriate time.** We can show bean dependencies to Spring without handling the setup and instantiation of those objects.

AD

The ability to use annotations like `@Autowired` (/spring-autowire) to inject Spring-managed beans into our application is a driving force for creating powerful and scalable code in Spring. (https://freestar.c
utm_campaign=branding&utm_medium=banner&utm_source=bael

So, how do we tell Spring about the beans we want it to manage for us? **We should take advantage of Spring's automatic bean detection by using stereotype annotations in our classes.**

3. @Component

`@Component` is an annotation that allows Spring to detect our custom beans automatically.

In other words, without having to write any explicit code, Spring will:

- Scan our application for classes annotated with `@Component`
- Instantiate them and inject any specified dependencies into them
- Inject them wherever needed

However, most developers prefer to use more specialized stereotype annotations to serve this function.

AD

(https://freestar.c
utm_campaign=branding&utm_medium=banner&utm_source=bael

3.1. Spring Stereotype Annotations

Spring has provided a few specialized stereotype annotations: `@Controller`, `@Service` and `@Repository`. They all provide the same function as `@Component`.

They all act the same because they are all composed annotations with `@Component` as a meta-annotation for each of them. They are like `@Component` aliases with specialized uses and meaning outside Spring auto-detection or dependency injection.

We could theoretically use `@Component` exclusively for our bean auto-detection needs if we wanted to. On the flip side, we could also compose our specialized annotations (/java-custom-annotation) that use `@Component`.

However, other areas of Spring look specifically for Spring's specialized annotations to provide additional automation benefits. **So, we should probably stick with using the established specializations most of the time.**

Let's assume we have an example of each of these cases in our Spring Boot project:

AD

(https://freestar.c
utm_campaign=branding&utm_medium=banner&utm_source=bael

```

@Controller
public class ControllerExample {
}

@Service
public class ServiceExample {
}

@Repository
public class RepositoryExample {
}

@Component
public class ComponentExample {
}

@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Component
public @interface CustomComponent {
}

@CustomComponent
public class CustomComponentExample {
}

```

We could write a test that proves that each one is auto-detected by Spring and added to the *ApplicationContext*:

```

@SpringBootTest
@ExtendWith(SpringExtension.class)
public class ComponentUnitTest {

    @Autowired
    private ApplicationContext applicationContext;

    @Test
    public void givenInScopeComponents_whenSearchingInApplicationContext_thenFindThem() {
        assertNotNull(applicationContext.getBean(ControllerExample.class));
        assertNotNull(applicationContext.getBean(ServiceExample.class));
        assertNotNull(applicationContext.getBean(RepositoryExample.class));
        assertNotNull(applicationContext.getBean(ComponentExample.class));
        assertNotNull(applicationContext.getBean(CustomComponentExample.class));
    }
}

```

3.2. @ComponentScan

Before we rely entirely on *@Component*, we must understand that it's only a plain annotation. The annotation serves the purpose of differentiating beans from other objects, such as domain objects.

However, **Spring uses the *@ComponentScan* annotation to gather them into its *ApplicationContext*.**

If we're writing a Spring Boot application, it is helpful to know that *@SpringBootApplication* is a composed annotation that includes *@ComponentScan*. As long as our *@SpringBootApplication* class is at the root of our project, it will scan every *@Component* we define by default.

But in case our *@SpringBootApplication* class can't be at the root of our project or we want to scan outside sources, we can configure *@ComponentScan* (`/spring-component-scanning#component-scan`) explicitly to look in whatever package we specify, as long as it exists on the classpath.

Let's define an out-of-scope *@Component* bean:

```
package com.baeldung.component.scannedscope;

@Component
public class ScannedScopeExample {
}
```

Next, we can include it via explicit instructions to our `@ComponentScan` annotation:

```
package com.baeldung.component.inscope;

@SpringBootApplication
@ComponentScan({"com.baeldung.component.inscope", "com.baeldung.component.scannedscope"})
public class ComponentApplication {
    //public static void main(String[] args) {...}
}
```

Finally, we can test that it exists:

AD

```
@Test
public void givenScannedScopeComponent_whenSearchingInApplicationContext_thenFindIt() {
    assertNotNull(applicationContext.getBean(ScannedScopeExample.class));
}
```

This is more likely to happen when we want to scan for an outside dependency included in our project.

3.3. @Component Limitations

There are some scenarios where we want a specific object to become a Spring-managed bean when we can't use `@Component`.

Let's define an object annotated with `@Component` in a package outside of our project:

```
package com.baeldung.component.outsidescopescope;

@Component
public class OutsideScopeExample {
}
```

Here is a test that proves that the `ApplicationContext` does not include the outside component:

```
@Test
public void givenOutsideScopeComponent_whenSearchingInApplicationContext_thenFail() {
    assertThrows(NoSuchBeanDefinitionException.class, () ->
        applicationContext.getBean(OutsideScopeExample.class));
}
```

Also, we may not have access to the source code because it comes from a third-party source, and we're unable to add the `@Component` annotation. Or perhaps we want to conditionally use one bean implementation over another, depending on the environment we're running in. **Auto-detection is usually sufficient, but when it's not, we can use `@Bean`.**

4. @Component vs @Bean

@Bean is also an annotation that Spring uses to gather beans at runtime, but it's not used at the class level. Instead, **we annotate methods with *@Bean* so that Spring can store the method's result as a Spring bean.**

We'll first create a POJO that has no annotations:

```
public class BeanExample {
}
```

Inside our class annotated with *@Configuration*, we can create a bean-generating method:

AD

```
@Bean
public BeanExample beanExample() {
    return new BeanExample();
}
```

(https://freestar.com,utm_campaign=branding&utm_medium=banner&utm_source=baeldung)

BeanExample might represent a local class, or it might be an external class. It doesn't matter because we need to return an instance of it.

We can then write a test that verifies Spring did pick up the bean:

```
@Test
public void givenBeanComponents_whenSearchingInApplicationContext_thenFindThem() {
    assertNotNull(applicationContext.getBean(BeanExample.class));
}
```

We should note some important implications because of the differences between *@Component* and *@Bean*.

- *@Component* is a class-level annotation, but *@Bean* is at the method level, so *@Component* is only an option when a class's source code is editable. *@Bean* can always be used, but it's more verbose.
- *@Component* is compatible with Spring's auto-detection, but *@Bean* requires manual class instantiation.
- Using *@Bean*, decouples the instantiation of the bean from its class definition. This is why we can use it to make third-party classes into Spring beans. It also means we can introduce logic to decide which of several possible instance options for a bean to use.

5. Conclusion

We've just explored the Spring *@Component* annotation and other relevant topics. First, we discussed the various Spring stereotype annotations, which are just specialized versions of *@Component*.

Then we learned that *@Component* doesn't do anything unless *@ComponentScan* can find it.

Finally, since it's not possible to use *@Component* on classes because we don't have the source code, we learned how to use the *@Bean* annotation instead.

All of these code examples and more can be found on GitHub (<https://github.com/eugenp/tutorials/tree/master/spring-di-4>).

Get started with Spring 5 and Spring Boot 2, through the *Learn Spring* course:

>> THE COURSE (/ls-course-end)



Learning to build your API with Spring?

Download the E-book (/rest-api-spring-guide)

Comments are closed on this article!

AD

(<https://freestar.com>
utm_campaign=branding&utm_medium=banner&utm_source=bael
tf_2)

COURSES

[ALL COURSES \(/ALL-COURSES\)](#)

[ALL BULK COURSES \(/ALL-BULK-COURSES\)](#)

[ALL BULK TEAM COURSES \(/ALL-BULK-TEAM-COURSES\)](#)

[THE COURSES PLATFORM \(HTTPS://COURSES.BAELDUNG.COM\)](https://courses.baeldung.com)

SERIES

[JAVA "BACK TO BASICS" TUTORIAL \(/JAVA-TUTORIAL\)](#)

[JACKSON JSON TUTORIAL \(/JACKSON\)](#)

[APACHE HTTPCLIENT TUTORIAL \(/HTTPCLIENT-GUIDE\)](#)

[REST WITH SPRING TUTORIAL \(/REST-WITH-SPRING-SERIES\)](#)

[SPRING PERSISTENCE TUTORIAL \(/PERSISTENCE-WITH-SPRING-SERIES\)](#)

[SECURITY WITH SPRING \(/SECURITY-SPRING\)](#)

[SPRING REACTIVE TUTORIALS \(/SPRING-REACTIVE-GUIDE\)](#)

ABOUT

[ABOUT BAELDUNG \(/ABOUT\)](#)

[THE FULL ARCHIVE \(/FULL_ARCHIVE\)](#)

[EDITORS \(/EDITORS\)](#)

[JOBS \(/TAG/ACTIVE-JOB/\)](#)

[OUR PARTNERS \(/PARTNERS\)](#)

[PARTNER WITH BAELDUNG \(/ADVERTISE\)](#)

[TERMS OF SERVICE \(/TERMS-OF-SERVICE\)](#)

[PRIVACY POLICY \(/PRIVACY-POLICY\)](#)

[COMPANY INFO \(/BAELDUNG-COMPANY-INFO\)](#)

[CONTACT \(/CONTACT\)](#)